# 🐍 Python Data Types Complete Documentation (With Java Comparison)

---

## 🌟 What are Data Types?

Data types define:

- What kind of value a variable stores
- How memory is allocated
- What operations can be performed on the data

---

## 🔥 Example

```
x = 10
name = "Python"
price = 99.5
```

Here:

- `10` → Integer
- `"Python"` → String
- `99.5` → Float

---

## ☕ Java vs 🐍 Python Data Types

| Feature | Java | Python |
|---|---|---|
| Typing | Static Typing | Dynamic Typing |
| Datatype Declaration | Mandatory | Not Required |
| Primitive Types | Yes | No Separate Primitive Types |
| Object-Based | Partially | Everything is Object |
| Memory Management | JVM | Python Interpreter |
| Flexibility | Less | More |

---

# 🧠 Dynamic Typing in Python

```
x = 10
print(type(x))

x = "Hello"
print(type(x))
```

**Output**

```
<class 'int'>
<class 'str'>
```

👉 Same variable can hold different datatypes.

---

# 📚 Main Data Types in Python

```
Data Types
│
├── Numeric Types
│   ├── int
│   ├── float
│   └── complex
│
├── Sequence Types
│   ├── str
│   ├── list
│   ├── tuple
│   └── range
│
├── Set Types
│   ├── set
│   └── frozenset
│
├── Mapping Type
│   └── dict
│
├── Boolean Type
│   └── bool
│
```

```
└── Binary Types
    ├── bytes
    ├── bytearray
    └── memoryview
```

---

# 🔢 1️⃣ INTEGER (int)

---

## 📘 What is int?

Integer stores whole numbers.

---

## ✍️ Syntax

```
x = 10
```

---

## ✅ Examples

```
age = 21
marks = -100
salary = 50000
```

---

## ☕ Java Equivalent

```
int x = 10;
long y = 999999999;
```

---

## 🌍 Range

Python integers have:

✅ Unlimited size (depends on memory)

---

# ☕ Java Range

| Type | Range |
|---|---|
| int | -2^31 to 2^31-1 |
| long | Very large fixed range |

👉 Python integers are more flexible.

---

# 🔓 Mutability

❌ Immutable

---

# 🧪 Example

```
x = 10
print(id(x))

x = x + 1
print(id(x))
```

👉 New object is created.

---

# 🧠 Integer Interning

Python stores small integers in cache.

Usually:

```
-5 to 256
```

---

## Example

```
x = 100
y = 100

print(x is y)
```

**Output**

```
True
```

👉 Both point to same memory.

---

# 🔢 2️⃣ FLOAT

---

# 📘 What is float?

Stores decimal values.

---

# ✍️ Syntax

```
price = 99.5
```

---

# ✅ Example

```
pi = 3.14
salary = 9999.99
```

---

## ☕ Java Equivalent

```
double d = 99.5;
float f = 10.5f;
```

---

## 🔒 Mutability

❌ Immutable

---

## 🌍 Range

Very large floating point values supported.

---

## ⚠️ Floating Point Precision Issue

```
print(0.1 + 0.2)
```

**Output**

```
0.30000000000000004
```

👉 Happens because of binary representation.

---

## 🔢 3️⃣ COMPLEX

---

## 📘 What is complex?

Stores imaginary numbers.

---

## ✍️ Syntax

```
x = 2 + 3j
```

---

## ✅ Example

```
num = 5 + 6j

print(num.real)
print(num.imag)
```

---

## ☕ Java Equivalent

❌ No built-in complex datatype.

Need libraries.

---

## 🔒 Mutability

❌ Immutable

---

## 🔤 4️⃣ STRING (str)

---

## 📘 What is String?

String stores text data.

---

## ✍️ Syntax

```
name = "Python"
```

---

## ✅ Examples

```
city = 'Bangalore'
course = "Python"
```

---

## ☕ Java Equivalent

```
String name = "Python";
```

---

## 🔒 Mutability

❌ Immutable

---

## 🧠 String Interning

Python internally reuses common strings.

---

## Example

```
a = "hello"
b = "hello"

print(a is b)
```

**Output**

```
True
```

---

# ⚠️ String Immutability Example

```
name = "Python"

# name[0] = 'J'
```

👉 Error.

---

# ✅ Correct Way

```
name = "J" + name[1:]
```

---

# 🌍 Multiline Strings

```
text = """
Hello
World
"""
```

---

# 🧪 Common Operations

```
name.upper()
name.lower()
len(name)
```

---

# 📋 5️⃣ LIST

---

## 📘 What is List?

Stores multiple values.

---

## ✍️ Syntax

```
nums = [1, 2, 3]
```

---

## ☕ Java Equivalent

```
ArrayList<Integer>
```

---

## 🔒 Mutability

✅ Mutable

---

## ✅ Example

```
nums.append(4)
nums.remove(2)
```

---

## ⚠️ Mixed Datatypes Allowed

```
values = [1, "Hello", True]
```

---

# 📦 6️⃣ TUPLE

---

## 📘 What is Tuple?

Immutable collection.

---

## ✍️ Syntax

```
data = (1, 2, 3)
```

---

## 🔒 Mutability

❌ Immutable

---

## ⚠️ Important Edge Case

```
x = (5)
print(type(x))
```

**Output**

```
<class 'int'>
```

---

## ✅ Correct Single Element Tuple

```
x = (5,)
```

---

## ☕ Java Equivalent

No exact equivalent.

---

## 🧩 7️⃣ SET

---

## 📘 What is Set?

Stores unique values.

---

## ✍️ Syntax

```
nums = {1, 2, 3}
```

---

## 🔒 Mutability

✅ Mutable

---

## ⚠️ Duplicates Removed Automatically

```
nums = {1,1,1,2}
print(nums)
```

**Output**

```
{1, 2}
```

---

# ⚠️ Empty Set Problem

```
s = {}
print(type(s))
```

**Output**

```
<class 'dict'>
```

---

# ✅ Correct Empty Set

```
s = set()
```

---

# ☕ Java Equivalent

```
HashSet
```

---

# 📖 8️⃣ DICTIONARY (dict)

---

# 📘 What is Dictionary?

Stores data in:

```
key : value
```

format.

---

## ✍️ Syntax

```
student = {
    "name": "Rahul",
    "age": 21
}
```

---

## 🔒 Mutability

✅ Mutable

---

## ☕ Java Equivalent

```
HashMap
```

---

## ⚠️ Duplicate Keys Not Allowed

```
x = {
    "a": 1,
    "a": 2
}

print(x)
```

**Output**

```
{'a': 2}
```

---

## ⚠️ Dictionary Keys Must Be Immutable

✅ Allowed:

```
{
    1: "Hello",
    (1,2): "Tuple"
}
```

❌ Not Allowed:

```
{
    [1,2]: "List"
}
```

---

# ✅ 9️⃣ BOOLEAN (bool)

---

## 📘 What is Boolean?

Stores:

- True
- False

---

## ✍️ Syntax

```
is_active = True
```

---

## ☕ Java Equivalent

```
boolean isActive = true;
```

---

## 🔒 Mutability

❌ Immutable

---

## ⚠️ Important Point

```
print(True == 1)
print(False == 0)
```

**Output**

```
True
True
```

---

## 📏 🔟 RANGE

---

## 📘 What is range?

Used for sequences of numbers.

Mostly used in loops.

---

## ✍️ Syntax

```
range(start, stop, step)
```

---

## ✅ Example

```
for i in range(1, 10, 2):
    print(i)
```

---

## ☕ Java Equivalent

No direct equivalent.

Closest:

```
for(int i=0; i<10; i++)
```

---

## 🔓 Mutability

❌ Immutable

---

## 💾 Binary Data Types

---

### bytes

Immutable binary data.

```
x = b"hello"
```

---

### bytearray

Mutable binary data.

```
x = bytearray(5)
```

---

### memoryview

Used for memory-efficient binary access.

---

# 🔥 Mutable vs Immutable Data Types

| Mutable | Immutable |
|---|---|
| list | int |
| set | float |
| dict | bool |
| bytearray | str |
| | tuple |
| | range |
| | bytes |
| | frozenset |

---

# 🌟 Type Conversion

---

## Integer to String

```
x = str(10)
```

---

## String to Integer

```
x = int("100")
```

---

## Integer to Float

```
x = float(10)
```

---

# ⚠️ Invalid Conversion

```
int("Hello")
```

👉 Error.

---

# 🧠 Memory Concepts

---

## id() Function

Returns memory address.

```
x = 10
print(id(x))
```

---

## is Operator

Checks memory reference.

```
x = 100
y = 100

print(x is y)
```

---

## == Operator

Checks values.

```
x = [1,2]
y = [1,2]

print(x == y)
```

## is vs ==

| Operator | Meaning |
|---|---|
| `==` | Compare values |
| `is` | Compare memory location |

## Example

```
x = [1,2]
y = [1,2]

print(x == y)
print(x is y)
```

**Output**

```
True
False
```

## 🚀 Where Different Datatypes are Used

| Datatype | Used For |
|---|---|
| int | Counting, IDs |
| float | Prices, percentages |
| complex | Scientific calculations |
| str | Names, messages |
| list | Dynamic collections |
| tuple | Fixed data |
| set | Unique values |
| dict | Key-value storage |

| Datatype | Used For |
|---|---|
| bool | Conditions |
| range | Loops |

---

# 🎯 Final Summary

## Python Data Types are:

✅ Dynamic

✅ Object-based

✅ Flexible

✅ Easy to use

---

# 🧠 Most Important Interview Points

✅ Everything in Python is object

✅ Python uses dynamic typing

✅ Strings are immutable

✅ Lists are mutable

✅ Tuples are immutable

✅ Sets remove duplicates

✅ `{}` creates dictionary

✅ `(5)` is int, `(5,)` is tuple

✅ Small integers are internally cached

✅ `==` checks values

✅ `is` checks memory location